10 Managing data

This chapter covers

- Examining the lifecycle and states of objects
- Working with the EntityManager interface
- Working with detached state

You now understand how ORM solves the static aspects of the object/relational mismatch. With what you know so far, you can create a mapping between Java classes and an SQL schema, solving the structural mismatch problem. As you'll recall, the paradigm mismatch covers the problems of granularity, inheritance, identity, association, and data navigation. For a deeper review, take a look back at section 1.2.

Beyond that, though, an efficient application solution requires something more: you must investigate strategies for runtime data management. These strategies are crucial to the performance and correct behavior of the applications.

In this chapter, we'll analyze the lifecycle of entity instances—how an instance becomes persistent, and how it stops being considered persistent—and the method calls and management operations that trigger these transitions. The JPA `EntityManager` is the primary interface for accessing data.

Before we look at JPA, let's start with entity instances, their lifecycle, and the events that trigger a change of state. Although some of this material may be formal, a solid understanding of the persistence lifecycle is essential.

Major new features in JPA 2

We can get a vendor-specific variation of the persistence manager API with `EntityManager#unwrap()`, such as the `org.hibernate.Session` API. Use the already demonstrated `EntityManagerFactory#unwrap()` method to obtain an instance of `org.hibernate.SessionFactory` (see section 2.5).

The new `detach()` operation provides fine-grained management of the persistence context, evicting individual entity instances.

From an existing `EntityManager`, we can obtain the `EntityManagerFactory` used to create the persistence context with

`getEntityManagerFactory()`.

The new static `PersistenceUtil` and `PersistenceUnitUtil` helper methods determine whether an entity instance (or one of its properties) was fully loaded or is an uninitialized reference (a Hibernate proxy or an unloaded collection wrapper).

10.1 The persistence lifecycle

Because JPA is a transparent persistence mechanism, where classes are unaware of their own persistence capability, it is possible to write application logic that is unaware of whether the data it operates on represents a persistent state or a temporary state that exists only in memory. The application shouldn't necessarily need to care that an instance is persistent when invoking its methods. We can, for example, invoke the `Item#calculateTotalPrice()` business method without having to consider persistence at all (such as in a unit test). The method may be unaware of any persistence concept while its executing.

Any application with a persistent state must interact with the persistence service whenever it needs to propagate the state held in memory to the database (or vice versa). In other words, we have to call the Jakarta Persistence interfaces to store and load data.

When interacting with the persistence mechanism that way, the application must concern itself with the state and lifecycle of an entity instance with respect to persistence. We refer to this as the *persistence lifecycle* : the states an entity instance goes through during its life, and we'll analyze them in a moment. We also use the term *unit of work*: a set of (possibly) state-changing operations considered one (usually atomic) group.
Another piece of the puzzle is the *persistence context* provided by the persistence service. Think of the persistence context as a service that remembers all the modifications and state changes we made to the data in a particular unit of work (this is somewhat simplified, but it's a good starting point).

We'll now dissect the following terms: *entity states*, *persistence contexts*, and *managed scope*. You're probably more accustomed to thinking about which SQL statements you have to manage to get stuff in and out of the database, but one of the key factors of the success of Java Persistence is the analysis of *state management*, so stick with us through this section.

10.1.1 Entity instance states

Different ORM solutions use different terminology and define different states and state transitions for the persistence lifecycle. Moreover, the states used internally may be different from those exposed to the client application. JPA defines four states, hiding the complexity of Hibernate's internal implementation from the client code. Figure 10.1 shows these states and their transitions.

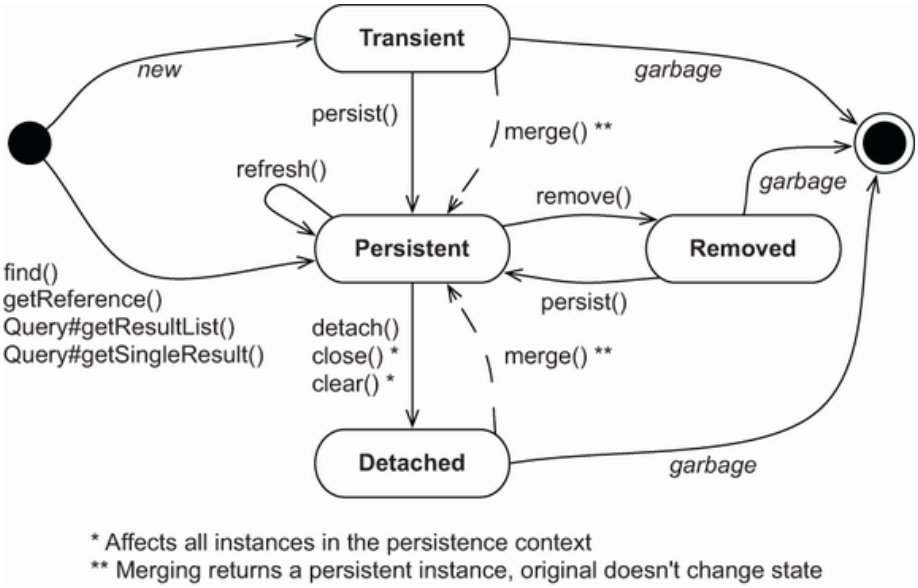

Figure 10.1 Entity instance states and their transitions

Figure 10.1 also includes the method calls to the `EntityManager` (and `Query`) API that triggers transitions. We'll discuss this chart in this chapter; refer to it whenever you need an overview.

Now, let's explore the states and transitions in more detail.

Transient state

Instances created with the `new` Java operator are *transient*, which means their state is lost and garbage-collected as soon as they're no longer referenced. For example, `new Item()` creates a transient instance of the `Item` class, just like `new Long ()` and `new BigDecimal()` create transient instances of those classes. Hibernate doesn't provide any rollback functionality for transient instances; if we modify the price of a transient `Item`, we can't automatically undo the change.

For an entity instance to transition from transient to *persistent* state requires either a call to the `EntityManager#persist()` method or the creation of a reference from an already-persistent instance and enabled cascading of state for that mapped association.

Persistent state

A *persistent* entity instance has a representation in the database. It's stored in the database—or it will be stored when the unit of work completes. It's an instance with a database identity, as defined in section 5.2; its database identifier is set to the primary key value of the database representation.

The application may have created instances and then made them persistent by calling `EntityManager#persist()`. Instances may have also become persistent when the application created a reference to the object from another persistent instance that the JPA provider already manages. A persistent entity instance may be an instance retrieved from the data-

base by executing a query, an identifier lookup, or navigating the object graph starting from another persistent instance.

Persistent instances are always associated with a persistence context. We'll see more about this in a moment.

Removed state

We can delete a persistent entity instance from the database in several ways. For example, we can remove it with `EntityManager#remove()`. It may also become available for deletion if we remove a reference to it from a mapped collection with *orphan removal* enabled.

An entity instance is then in the *removed* state: the provider will delete it at the end of a unit of work. We should discard any references we may hold to it in the application after we finish working with it—for example, after we've rendered the removal-confirmation screen the users see.

Detached state

To understand *detached* entity instances, consider loading an instance. We call `EntityManager#find()` to retrieve an entity instance by its (known) identifier. Then we end our unit of work and close the persistence context. The application still has a *handle*—a reference to the instance we loaded. It's now in a detached state, and the data is becoming stale. We could discard the reference and let the garbage collector reclaim the memory. Or, we could continue working with the data in the detached state and later call the `merge()` method to save our modifications in a new unit of work. We'll discuss detachment and merging in section 10.3.

You should now have a basic understanding of entity instance states and their transitions. Our next topic is the persistence context: an essential service of any Jakarta Persistence provider.

10.1.2 The persistence context

In a Java Persistence application, an `EntityManager` has a persistence context. We create a persistence context when we call `EntityManagerFactory#createEntityManager()`. The context is closed when we call `EntityManager#close()`. In JPA terminology, this is an *application-managed* persistence context; our application defines the scope of the persistence context, demarcating the unit of work.

The persistence context monitors and manages all entities in the persistent state. The persistence context is the centerpiece of much of the functionality of a JPA provider.

The persistence context also allows the persistence engine to perform *automatic dirty checking*, detecting which entity instances the application modified. The provider then synchronizes with the database the state of

instances monitored by a persistence context, either automatically or on demand. Typically, when a unit of work completes, the provider propagates state that's held in memory to the database through the execution of SQL `INSERT`, `UPDATE`, and `DELETE` statements (all part of the Data Manipulation Language, DML). This *flushing* procedure may also occur at other times. For example, Hibernate may synchronize with the database before the execution of a query. This ensures that queries are aware of changes made earlier during the unit of work.

The persistence context also acts as a *first-level cache*; it remembers all entity instances handled in a particular unit of work. For example, if we ask Hibernate to load an entity instance using a primary key value (a lookup by identifier), Hibernate can first check the current unit of work in the persistence context. If Hibernate finds the entity instance in the persistence context, no database hit occurs—this is a repeatable read for an application. Consecutive `em.find(Item.class, ITEM_ID)` calls with the same persistence context will yield the same result.

This cache also affects the results of arbitrary queries, such as those executed with the `javax.persistence.Query` API. Hibernate reads the SQL result set of a query and transforms it into entity instances. This process first tries to resolve every entity instance in the persistence context by identifier lookup. Only if an instance with the same identifier value can't be found in the current persistence context does Hibernate read the rest of the data from the result-set row. Hibernate ignores any potentially newer data in the result set, due to read-committed transaction isolation at the database level, if the entity instance is already present in the persistence context.

The persistence context cache is always on—it can't be turned off. It ensures the following:

- The persistence layer isn't vulnerable to stack overflows in the case of circular references in an object graph.
- There can never be conflicting representations of the same database row at the end of a unit of work. The provider can safely write all changes made to an entity instance to the database.
- Likewise, changes made in a particular persistence context are always immediately visible to all other code executed inside that unit of work and its persistence context. JPA guarantees repeatable entity-instance reads.

The persistence context provides a *guaranteed scope of object identity*; in the scope of a single persistence context, only one instance represents a particular database row. Consider the comparison of references `entityA == entityB`. This is `true` only if both are references to the same Java instance on the heap. Now consider the comparison `entityA.getId().equals(entityB.getId())`. This is `true` if both have the same database identifier value. Within one persistence context,

Hibernate guarantees that both comparisons will yield the same result. This solves one of the fundamental object/relational mismatch problems we discussed in section 1.2.3.

The lifecycle of entity instances and the services provided by the persistence context can be difficult to understand at first. Let's look at some code examples of dirty checking, caching, and how the guaranteed identity scope works in practice. To do this, we'll work with the persistence manager API.

Would process-scoped identity be better?

For a typical web or enterprise application, persistence context-scoped identity is preferred. Process-scoped identity, where only one in-memory instance represents the row in the entire process (JVM), offers some potential advantages in terms of cache utilization. In a pervasively multithreaded application, though, the cost of always synchronizing shared access to persistent instances in a global identity map is too high a price to pay. It's simpler and more scalable to have each thread work with a distinct copy of the data in each persistence context.

10.2 The EntityManager interface

Any transparent persistence tool includes a persistence manager API. This persistence manager usually provides services for basic CRUD (create, read, update, delete) operations, query execution, and controlling the persistence context. In Jakarta Persistence applications, the main interface we interact with is the `EntityManager` to create units of work.

NOTE To execute the examples from the source code, you'll first need to run the Ch10.sql script. The source code that follows can be found in the managing-data and managing-data2 folders.

We will not use Spring Data JPA in this chapter, not even the Spring framework. The examples that follow will use JPA and, sometimes, the Hibernate API, without any Spring integration—they are finer-grained for our demonstrations and analysis.

10.2.1 The canonical unit of work

In Java SE and some EE architectures (if we only have plain servlets, for example), we get an `EntityManager` by calling `EntityManagerFactory#createEntityManager()`. The application code shares the `EntityManagerFactory`, representing one persistence unit, or one logical database. Most applications have only one shared `EntityManagerFactory`.

We use the `EntityManager` for a single unit of work in a single thread, and it's inexpensive to create. The following listing shows the canonical, typical form of a unit of work.

Listing 10.1 A typical unit of work

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java

EntityManagerFactory emf =
        Persistence.createEntityManagerFactory("ch10");

// . . .
EntityManager em = emf.createEntityManager();

\1 {
    em.getTransaction().begin();

    // . . .
    em.getTransaction().commit();
} catch (Exception ex) {
    // Transaction rollback, exception handling
    // . . .
} finally {
    if (em != null && em.isOpen())
        em.close();
}
```

Everything between `em.getTransaction().begin()` and `em.get-Transaction() .commit()` occurs in one transaction. For now, keep in mind that all database operations in a transaction scope, such as the SQL statements executed by Hibernate, either completely succeed or completely fail. Don't worry too much about the transaction code for now; you'll read more about concurrency control in the next chapter. We'll look at the same example there with a focus on the transaction and exception- handling code. Don't write empty `catch` clauses in the code, though—you'll have to roll back the transaction and handle exceptions.

Creating an `EntityManager` starts its persistence context. Hibernate won't access the database until necessary; the `EntityManager` doesn't obtain a JDBC `Connection` from the pool until SQL statements have to be executed. We can even create and close an `EntityManager` without hitting the database. Hibernate executes SQL statements when we look up or query data and when it flushes changes detected by the persistence context to the database. Hibernate joins the in-progress system transaction when an `EntityManager` is created and waits for the transaction to commit. When Hibernate is notified of the commit, it performs dirty checking of the persistence context and synchronizes with the database. We can also force dirty checking synchronization manually by calling `EntityManager#flush()` at any time during a transaction.

We determine the scope of the persistence context by choosing when to `close()` the `EntityManager` . We have to close the persistence context at some point, so always place the `close()` call in a `finally` block.

How long should the persistence context be open? Let's assume for the following examples that we're writing a server, and each client request will be processed with one persistence context and system transaction in a multithreaded environment. If you're familiar with servlets, imagine the code in listing 10.1 embedded in a servlet's `service()` method. Within this unit of work, you access the `EntityManager` to load and store data.

10.2.2 Making data persistent

Let's create a new instance of an entity and bring it from transient into persistent state. You will do this whenever you want to save the information from a newly created object to the database. We can see the same unit of work and how the `Item` instances change state in figure 10.2.

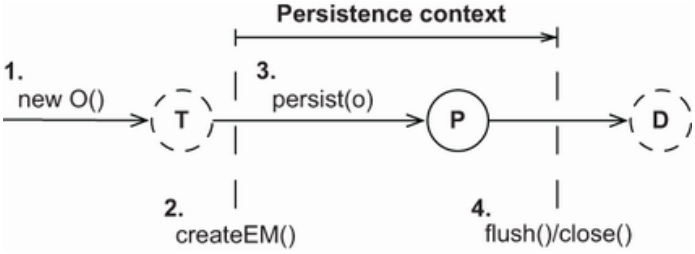

Figure 10.2 Making an instance persistent in a unit of work

To make an instance persistent, you can use a piece of code like the following:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java – makePersistent()

Item item = new Item();
item.setName("Some Item");
em.persist(item);
Long ITEM_ID = item.getId();
```

A new transient `Item` is instantiated as usual. Of course, we could also instantiate it before creating the `EntityManager`. A call to `persist()` makes the transient instance of `Item` persistent. It's then managed by and associated with the current persistence context.

To store the `Item` instance in the database, Hibernate has to execute an SQL `INSERT` statement. When the transaction of this unit of work commits, Hibernate flushes the persistence context, and the `INSERT` occurs at that time. Hibernate may even batch the `INSERT` at the JDBC level with other statements. When we call `persist()`, only the identifier value of the `Item` is assigned. Alternatively, if the identifier generator isn't *pre-insert*, the `INSERT` statement will be executed immediately when `persist()` is called. You may want to review section 5.2.5 for a refresher on the identifier generator strategies.

Detecting entity state using the identifier

Sometimes we need to know whether an entity instance is persistent, transient, or detached.

- *Persistent*—An entity instance is in persistent state if `EntityManager# contains(e)` returns `true`.
- *Transient*—It's in transient state if `PersistenceUnitUtil#getIdentifier(e)` returns `null`.
- *Detached*—It's in the detached state if it's not persistent, and `PersistenceUnitUtil#getIdentifier(e)` returns the value of the entity's identifier property.

We can get to `PersistenceUnitUtil` from the `EntityManagerFactory`.

There are two concerns to look out for. First, be aware that the identifier value may not be assigned and available until the persistence context is flushed. Second, Hibernate (unlike some other JPA providers) never returns `null` from `PersistenceUnitUtil#getIdentifier()` if the identifier property is a primitive (a `long` and not a `Long`).

It's better (but not required) to fully initialize the `Item` instance before managing it with a persistence context. The SQL `INSERT` statement contains the values that were held by the instance at the point when `persist()` was called. If we don't set the `name` of the `Item` before making it persistent, a `NOT NULL` constraint may be violated. We can modify the `Item` after calling `persist()`, and the changes will be propagated to the database with an additional SQL `UPDATE` statement.

If one of the `INSERT` or `UPDATE` statements fails during flushing, Hibernate causes a rollback of changes made to persistent instances in this transaction at the database level. But Hibernate doesn't roll back in-memory changes to persistent instances. If we change the `Item#name` after `persist()`, a commit failure won't roll back to the old name. This is reasonable, because a failure of a transaction is normally non-recoverable, and we have to discard the failed persistence context and `EntityManager` immediately. We'll discuss exception handling in the next chapter.

Next, we'll load and modify the stored data.

10.2.3 Retrieving and modifying persistent data

We can retrieve persistent instances from the database with the `EntityManager`. In a real-life use case, we would have kept the identifier value of the `Item` stored in the previous section somewhere, and we are now looking up the same instance in a new unit of work by identifier. Figure 10.3 shows this transition graphically.

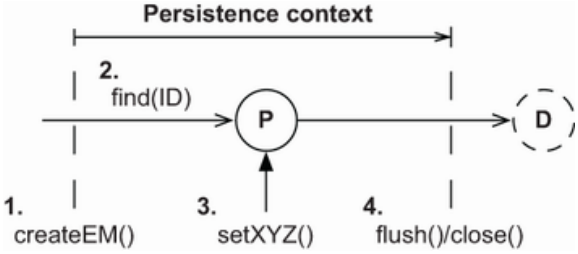

Figure 10.3 Making an instance persistent in a unit of work

To make an instance persistent in a unit of work, you can use a piece of code like the following:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java – retrievePersistent()

Item item = em.find(Item.class, ITEM_ID);     Ⓐ
if (item != null)
    item.setName("New Name");                 Ⓑ
```

Ⓐ The instruction will hit the database if `item` is not already in the persistence context.

Ⓑ Then we modify the name.

We don't need to cast the returned value of the `find()` operation; it's a generified method, and its return type is set as a side effect of the first parameter. The retrieved entity instance is in a persistent state, and we can now modify it inside the unit of work.

If no persistent instance with the given identifier value can be found, `find()` returns `null`. The `find()` operation always hits the database if there is no hit for the given entity type and identifier in the persistence context cache. The entity instance is always initialized during loading. We can expect to have all of its values available later in a detached state, such as when rendering a screen after we close the persistence context. (Hibernate may not hit the database if its optional second-level cache is enabled.)

We can modify the `Item` instance, and the persistence context will detect these changes and record them in the database automatically. When Hibernate flushes the persistence context during commit, it executes the necessary SQL DML statements to synchronize the changes with the database. Hibernate propagates state changes to the database as late as possible, toward the end of the transaction. DML statements usually create locks in the database that are held until the transaction completes, so Hibernate keeps the lock duration in the database as short as possible.

Hibernate writes the new `Item#name` to the database with an SQL `UPDATE`. By default, Hibernate includes all columns of the mapped `ITEM` table in the SQL `UPDATE` statement, updating unchanged columns to

their old values. Hence, Hibernate can generate these basic SQL statements at startup, not at runtime. If we want to include only modified (or non-nullable for `INSERT`) columns in SQL statements, we can enable dynamic SQL generation as demonstrated in section 5.3.2.

Hibernate detects the changed `name` by comparing the `Item` with a snapshot copy it took when the `Item` was loaded from the database. If the `Item` is different from the snapshot, an `UPDATE` is necessary. This snapshot in the persistence context consumes memory. Dirty checking with snapshots can also be time-consuming because Hibernate has to compare all instances in the persistence context with their snapshots during flushing.

We mentioned earlier that the persistence context enables repeatable reads of entity instances and provides an object-identity guarantee:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java – retrievePersistent()

Item itemA = em.find(Item.class, ITEM_ID);      Ⓐ
Item itemB = em.find(Item.class, ITEM_ID);      Ⓑ
assertTrue(itemA == itemB);
assertTrue(itemA.equals(itemB));
assertTrue(itemA.getId().equals(itemB.getId()));
```

Ⓐ The first `find()` operation hits the database and retrieves the `Item` instance with a `SELECT` statement.

Ⓑ The second `find()` is a repeatable read and is resolved in the persistence context, and the same cached `Item` instance is returned.

Sometimes we need an entity instance but we don't want to hit the database.

10.2.4 Getting a reference

If we don't want to hit the database when loading an entity instance, because we aren't sure we need a fully initialized instance, we can tell the `EntityManager` to attempt the retrieval of a hollow placeholder—a proxy.

If the persistence context already contains an `Item` with the given identifier, that `Item` instance is returned by `getReference()` without hitting the database. Furthermore, if *no* persistent instance with that identifier is currently managed, Hibernate produces the hollow placeholder: the proxy. This means `getReference()` won't access the database, and it doesn't return `null`, unlike `find()`. JPA offers the `PersistenceUnitUtil` helper methods. The isLoaded() helper method is used to detect whether we're working with an uninitialized proxy.

As soon as we call any method, such as `Item#getName()`, on the proxy, a `SELECT` is executed to fully initialize the placeholder. The exception to this rule is a mapped database identifier getter method, such as `getId()`. A proxy may look like the real thing, but it's only a placeholder carrying the identifier value of the entity instance it represents. If the database record no longer exists when the proxy is initialized, an `EntityNotFoundException` is thrown. Note that the exception can be thrown when `Item#getName()` is called. The Hibernate class has a convenient static `initialize()` method that loads the proxy's data.

After the persistence context is closed, `item` is in a detached state. If we don't initialize the proxy while the persistence context is still open, we get a `LazyInitializationException` if we access the proxy, as demonstrated in the following code. We can't load data on demand once the persistence context is closed. The solution is simple: load the data before closing the persistence context.

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java — retrievePersistentReference()

Item item = em.getReference(Item.class, ITEM_ID);
PersistenceUnitUtil persistenceUtil =
    emf.getPersistenceUnitUtil();
assertFalse(persistenceUtil.isLoaded(item));
// assertEquals("Some Item", item.getName());
// Hibernate.initialize(item);
em.getTransaction().commit();
em.close();
assertThrows(LazyInitializationException.class, () -> item.getName());
```

Ⓐ The persistence context.

Ⓑ The helper methods.

Ⓒ Detecting an uninitialized proxy.

Ⓓ Mapping the exception to the rule.

Ⓔ Load the proxy data.

Ⓕ `item` is in a detached state.

Ⓖ Load data after closing the persistence context.

We'll have much more to say about proxies, lazy loading, and on-demand fetching in chapter 12.

Next, if we want to remove the state of an entity instance from the database, we have to make it transient.

10.2.5 Making data transient

To make an entity instance transient and delete its database representation, we can call the `remove()` method on the `EntityManager`. Figure 10.4 shows this process.

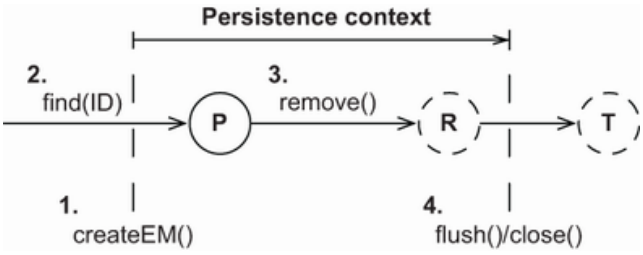

Figure 10.4 Removing an instance in a unit of work

If we call `find()`, Hibernate executes a `SELECT` to load the `Item`. If we call `getReference()`, Hibernate attempts to avoid the `SELECT` and returns a proxy. Calling `remove()` queues the entity instance for deletion when the unit of work completes; it's now in *removed* state. If `remove()` is called on a proxy, Hibernate executes a `SELECT` to load the data. An entity instance must be fully initialized during lifecycle transitions. We may have lifecycle callback methods or an entity listener enabled (see section 13.2), and the instance must pass through these interceptors to complete its full lifecycle.

An entity in a removed state is no longer in a persistent state. We can check this with the `contains()` operation. We can make the removed instance persistent again, canceling deletion.

When the transaction commits, Hibernate synchronizes the state transitions with the database and executes the SQL `DELETE`. The JVM garbage collector detects that the `item` is no longer referenced by anything and finally deletes the last trace of the data. We can finally close `EntityManager`:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java – makeTransient()

Item item = em.find(Item.class, ITEM_ID);      Ⓐ
em.remove(item);                               Ⓑ
assertFalse(em.contains(item));                Ⓒ
// em.persist(item);                           Ⓓ
assertNull(item.getId());                      Ⓔ
em.getTransaction().commit();                  Ⓕ
em.close();                                    Ⓖ
```

Ⓐ Call `find()`; Hibernate executes a `SELECT` to load the `Item`.

Ⓑ Call `remove()`; Hibernate queues the entity instance for deletion when the unit of work completes.

Ⓒ An entity in a removed state is no longer contained in the persistence context.

Ⓓ Canceling deletion makes the removed instance persistent again.

Ⓔ `item` will now look like a transient instance.

Ⓕ The transaction commits; Hibernate synchronizes the state transitions with the database and executes the SQL `DELETE`.

Ⓖ Close `EntityManager`.

By default, Hibernate won't alter the identifier value of a removed entity instance. This means the `item.getId()` method still returns the now outdated identifier value. Sometimes it's useful to work with the "deleted" data further: for example, we might want to save the removed `Item` again if our user decides to undo. As shown in the example, we can call `persist()` on a removed instance to cancel the deletion before the persistence context is flushed. Alternatively, if we set the property `hibernate.use_identifier_rollback` to `true` in persistence.xml, Hibernate will reset the identifier value after the removal of an entity instance. In the previous code example, the identifier value is reset to the default value of `null` (it's a `Long`). The `Item` is now the same as in a transient state, and we can save it again in a new persistence context.

Let's say we load an entity instance from the database and work with the data. For some reason, we know that another application or maybe another thread of the application has updated the underlying row in the database. Next we'll see how to refresh the data held in memory.

10.2.6 Refreshing data

It is possible that, after you have loaded an entity instance, some other process changes the information corresponding to the instance in the database. The following example demonstrates refreshing a persistent entity instance:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java – refresh()

Item item = em.find(Item.class, ITEM_ID);
item.setName("Some Name");
// Someone updates this row in the database with "Concurrent UpdateName"
em.refresh(item);
em.close();
assertEquals("Concurrent UpdateName", item.getName());
```

After we load the entity instance, we realize (it isn't important how) that someone else changed the data in the database. Calling `refresh()` causes Hibernate to execute a `SELECT` to read and marshal a whole result set,

overwriting changes we already made to the persistent instance in application memory. As a result, the `item`'s `name` is updated with the value set from the other side. If the database row no longer exists (if someone deleted it), Hibernate throws an `EntityNotFoundException` on `refresh()`.

Most applications don't have to manually refresh the in-memory state; concurrent modifications are typically resolved at transaction commit time. The best use case for refreshing is with an extended persistence context, which might span several request/response cycles or system transactions. While we wait for user input with an open persistence context, data gets stale, and selective refreshing may be required depending on the duration of the conversation and the dialogue between the user and the system. Refreshing can be useful to undo changes made in memory during a conversation if the user cancels the dialogue.

Another infrequently used operation is the replication of an entity instance.

10.2.7 Replicating data

Replication is useful, for example, when we need to retrieve data from one database and store it in another. Replication takes detached instances loaded in one persistence context and makes them persistent in another persistence context. We usually open these contexts from two different `EntityManagerFactory` configurations, enabling two logical databases. We have to map the entity in both configurations.

The `replicate()` operation is only available on the Hibernate `Session` API. Here is an example that loads an `Item` instance from one database and copies it into another:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java – replicate()

EntityManager emA = getDatabaseA().createEntityManager();
emA.getTransaction().begin();
Item item = emA.find(Item.class, ITEM_ID);
emA.getTransaction().commit();

EntityManager emB = getDatabaseB().createEntityManager();
emB.getTransaction().begin();
emB.unwrap(Session.class)
        .replicate(item, org.hibernate.ReplicationMode.LATEST_VERSION);
Item item1 = emB.find(Item.class, ITEM_ID);
assertEquals("Some Item", item1.getName());
emB.getTransaction().commit();
emA.close();
emB.close();
```

`ReplicationMode` controls the details of the replication procedure:

- `IGNORE` —Ignores the instance when there is an existing database row with the same identifier in the database.
- `OVERWRITE` —Overwrites any existing database row with the same identifier in the database.
- `EXCEPTION` —Throws an exception if there is an existing database row with the same identifier in the target database.
- `LATEST_VERSION` —Overwrites the row in the database if its version is older than the version of the given entity instance, or ignores the instance otherwise. Requires enabled optimistic concurrency control with entity versioning (discussed in section 11.2.2).

We may need replication when we reconcile data entered into different databases. One use case is a product upgrade: if the new version of the application requires a new database (schema), we may want to migrate and replicate the existing data once.

The persistence context does many things for you: automatic dirty checking, guaranteed scope of object identity, and so on. It's equally important that you know some of the details of its management, and that you sometimes influence what goes on behind the scenes.

10.2.8 Caching in the persistence context

The persistence context is a cache of persistent instances. Every entity instance in a persistent state is associated with the persistence context.

Many Hibernate users who ignore this simple fact run into an `OutOfMemoryError`. This is typically the case when we load thousands of entity instances in a unit of work but never intend to modify them. Hibernate still has to create a snapshot of each instance in the persistence context cache, which can lead to memory exhaustion. (Obviously, we should execute a bulk data operation if we modify thousands of rows.)

The persistence context cache never shrinks automatically, so you should keep the size of the persistence context to the minimum necessary. Often, many persistent instances in the context are there by accident—for example, because we needed only a few items but queried for many. Extremely large graphs can have a serious performance consequence and require significant memory for state snapshots. Check that queries return only the data you need, and consider the following ways to control Hibernate's caching behavior.

You can call `EntityManager#detach(i)` to evict a persistent instance manually from the persistence context. You can call `EntityManager#clear()` to detach all persistent entity instances, leaving you with an empty persistence context.

The native `Session` API has some extra operations you might find useful. You can set the entire persistence context to read-only mode. This dis-

ables state snapshots and dirty checking, and Hibernate won't write modifications to the database:

```
Path: managing-data2/src/test/java/com/manning/javapersistence/ch10
➥ /ReadOnly.java – selectiveReadOnly()

em.unwrap(Session.class).setDefaultReadOnly(true);      Ⓐ
Item item = em.find(Item.class, ITEM_ID);
item.setName("New Name");
em.flush();                                             Ⓑ
```

Ⓐ Set the persistence context to read-only.

Ⓑ Consequently, `flush()` will not update the database.

You can disable dirty checking for a single entity instance:

```
Path: managing-data2/src/test/java/com/manning/javapersistence/ch10
➥ /ReadOnly.java – selectiveReadOnly()

Item item = em.find(Item.class, ITEM_ID);
em.unwrap(Session.class).setReadOnly(item, true);     Ⓐ
item.setName("New Name");
em.flush();                                           Ⓑ
```

Ⓐ Set `item` in the persistence context to read-only.

Ⓑ Consequently, `flush()` will not update the database.

A query with the `org.hibernate.Query` interface can return read-only results, which Hibernate doesn't check for modifications:

```
Path: managing-data2/src/test/java/com/manning/javapersistence/ch10
➥ /ReadOnly.java – selectiveReadOnly()

org.hibernate.query.Query query = em.unwrap(Session.class)
    .createQuery("select i from Item i");
query.setReadOnly(true).list();                 Ⓐ
List<Item> result = query.list();
for (Item item : result)
    item.setName("New Name");
em.flush();                                     Ⓑ
```

Ⓐ Set the query to read-only.

Ⓑ Consequently, `flush()` will not update the database.

With query hints you can also disable dirty checking for instances obtained with the JPA standard `javax.persistence.Query` interface:

```
Query query = em.createQuery(queryString)
    .setHint(
        org.hibernate.annotations.QueryHints.READ_ONLY,
        true
    );
```

Be careful with read-only entity instances: you can still delete them, and modifications to collections are tricky! The Hibernate manual has a long list of special cases you need to read if you use these settings with mapped collections.

So far, flushing and synchronization of the persistence context have occurred automatically, when the transaction commits. In some cases, though, we need more control over the synchronization process.

10.2.9 Flushing the persistence context

By default, Hibernate flushes the persistence context of an `EntityManager` and synchronizes changes with the database whenever the joined transaction is committed. All the previous code examples, except some in the last section, have used that strategy. JPA allows implementations to synchronize the persistence context at other times if they wish.

Hibernate, as a JPA implementation, synchronizes at the following times:

- When a joined Java Transaction API (JTA) system transaction is committed.
- Before a query is executed—we don't mean a lookup with `find()` but a query with `javax.persistence.Query` or the similar Hibernate API.
- When the application calls `flush()` explicitly.

We can control this behavior with the `FlushModeType` setting of an `EntityManager`:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java — flushModeType()

em.getTransaction().begin();
Item item = em.find(Item.class, ITEM_ID);
item.setName("New Name");

em.setFlushMode(FlushModeType.COMMIT);

assertEquals(
        "Original Name",
        em.createQuery("select i.name from Item i where i.id = :id", Stri
                .setParameter("id", ITEM_ID).getSingleResult()
);
```

```
em.getTransaction().commit(); // Flush!
em.close();
```

Here, we load an `Item` instance and change its name. Then we query the database, retrieving the item's name. Usually, Hibernate recognizes that data has changed in memory and synchronizes these modifications with the database before the query. This is the behavior of `FlushModeType.AUTO`, the default if we join the `EntityManager` with a transaction. With `FlushModeType.COMMIT` we disable flushing before queries, so we may see different data returned by the query than what we have in memory. The synchronization then occurs only when the transaction commits.

We can, at any time while a transaction is in progress, force dirty checking and synchronization with the database by calling `EntityManager#flush()`.

This concludes our discussion of the *transient*, *persistent*, and *removed* entity states, and the basic usage of the `EntityManager` API. Mastering these state transitions and API methods is essential; every JPA application is built with these operations.

Next we'll look at the *detached* entity state. We already mentioned some problems we'll see when entity instances aren't associated with a persistence context anymore, such as disabled lazy initialization. Let's explore the detached state with some examples, so we know what to expect when we work with data outside of a persistence context.

10.3 Working with detached state

If a reference leaves the scope of guaranteed identity, we call it a *reference* to a *detached entity instance*. When the persistence context is closed, it no longer provides an identity-mapping service. You'll run into aliasing problems when you work with detached entity instances, so make sure you understand how to handle the identity of detached instances.

10.3.1 The identity of detached instances

If we look up data using the same database identifier value in the same persistence context, the result is two references to the same in-memory instance on the JVM heap. When different references are obtained from the same persistence context, they have the same Java identity. The references may be equal because by default `equals()` relies on Java identity comparison. They obviously have the same database identity. They reference the same instance, in persistent state, managed by the persistence context for that unit of work.

References are in a detached state when the first persistence context is closed. We may be dealing with instances that live outside of a guaran-

teed scope of object identity.

Listing 10.2 Guaranteed scope of object identity in Java Persistence

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java — scopeOfIdentity()

em = emf.createEntityManager();                    Ⓐ
em.getTransaction().begin();                       Ⓑ
Item a = em.find(Item.class, ITEM_ID);             Ⓒ
Item b = em.find(Item.class, ITEM_ID);             Ⓒ
assertTrue(a == b);                                Ⓓ
assertTrue(a.equals(b));                           Ⓔ
assertEquals(a.getId(), b.getId());                Ⓕ
em.getTransaction().commit();                      Ⓖ
em.close();                                        Ⓗ
em = emf.createEntityManager();
em.getTransaction().begin();
Item c = em.find(Item.class, ITEM_ID);
assertTrue(a != c);                                Ⓘ
assertFalse(a.equals(c));                          Ⓙ
assertEquals(a.getId(), c.getId());                Ⓚ
em.getTransaction().commit();
em.close();
```

Ⓐ Create a persistence context.

Ⓑ Begin the transaction.

Ⓒ Load some entity instances.

Ⓓ References `a` and `b` are obtained from the same persistence context; they have the same Java identity.

Ⓔ `equals()` relies on Java identity comparison.

Ⓕ `a` and `b` reference the same `Item` instance, in persistent state, managed by the persistence context for that unit of work.

Ⓖ Commit the transaction.

Ⓗ Close the persistence context. References `a` and `b` are in a detached state when the first persistence context is closed.

Ⓘ `a` and `c`, loaded in different persistence contexts, aren't identical.

Ⓙ `a.equals(c)` is also `false`, because the `equals()` method has not been overridden, which means it uses instance equality (==).

Ⓚ A test for database identity still returns `true`.

If we treat entity instances as equal in detached state, this can lead to problems. For example, consider the following extension of the code, after the second unit of work has ended:

```
em.close();
Set<Item> allItems = new HashSet<>();
allItems.add(a);
allItems.add(b);
allItems.add(c);
assertEquals(2, allItems.size());
```

This example adds all three references to a `Set`, and all are references to detached instances. Now if we check the size of the collection—the number of elements—what result should we expect?

A `Set` doesn't allow duplicate elements. Duplicates are detected by the `Set`; whenever we add a reference to a `HashSet`, the `Item#equals()` method is called automatically against all other elements already in the collection. If `equals()` returns `true` for any element already in the collection, the addition doesn't occur.

By default, all Java classes inherit the `equals()` method of `java.lang.Object`. This implementation uses a double-equals (`==`) comparison to check whether two references refer to the same in-memory instance on the Java heap.

You might guess that the number of elements in the collection will be 2. After all, `a` and `b` are references to the same in-memory instance; they were loaded in the same persistence context. We obtained reference `c` from another persistence context; it refers to a different instance on the heap. We have three references to two instances, but we know this only because we've seen the code that loaded the data. In a real application, we may not know that `a` and `b` are loaded in a different context than `c`. Furthermore, we might expect that the collection has exactly one element because `a`, `b`, and `c` represent the same database row, the same `Item`.

Whenever we work with instances in a detached state and test them for equality (usually in hash-based collections), we need to supply our own implementation of the `equals ()` and `hashCode()` methods for our mapped entity class. This is an important issue: if we don't work with entity instances in a detached state, no action is needed, and the default `equals()` implementation of `java.lang.Object` is fine. We'll rely on Hibernate's guaranteed scope of object identity within a persistence context. Even if we work with detached instances, if we never check whether they're equal, or we never put them in a `Set` or use them as keys in a `Map`, we don't have to worry. If all we do is render a detached `Item` on the screen, we aren't comparing it to anything.

Let's assume that we want to use detached instances and that we have to test them for equality with our own method.

10.3.2 Implementing equality methods

We can implement `equals()` and `hashCode()` methods several ways. Keep in mind that when we override `equals()`, we also need to override `hashCode()` so the two methods are consistent. If two instances are equal, they must have the same hash value.

A seemingly clever approach is to implement `equals()` to compare just the database identifier property, which is often a surrogate primary key value. Basically, if two `Item` instances have the same identifier returned by `getId()`, they must be the same. If `getId()` returns `null`, it must be a transient `Item` that hasn't been saved.

Unfortunately, this solution has one huge problem: identifier values aren't assigned by Hibernate until an instance becomes persistent. If a transient instance were added to a `Set` before being saved, then when we save it, its hash value would change while it's contained by the `Set`. This is contrary to the contract of `java.util.Set`, breaking the collection. In particular, this problem makes cascading persistent states useless for mapped associations based on sets. We strongly discourage database identifier equality.

To get to the solution that we recommend, you need to understand the notion of a *business key*. A business key is a property or some combination of properties, that is unique for each instance with the same database identity. Essentially, it's the natural key that we would use if we weren't using a surrogate primary key instead. Unlike a natural primary key, it isn't an absolute requirement that the business key never changes—as long as it changes rarely, that's enough.

We argue that essentially every entity class should have a business key, even if it includes all properties of the class (which would be appropriate for some immutable classes). If our users are looking at a list of items on the screen, how do they differentiate between items A, B, and C? The same property, or combination of properties, is our business key. The business key is what the user thinks of as uniquely identifying a particular record, whereas the surrogate key is what the application and database systems rely on. The business key property or properties are most likely constrained `UNIQUE` in our database schema.

Let's write custom equality methods for the `User` entity class; this is easier than comparing `Item` instances. For the `User` class, `username` is a great candidate business key. It's always required, it's unique with a database constraint, and it changes rarely, if ever.

Listing 10.3 Custom implementation of `User` equality

```
@Entity
@Table(name = "USERS",
       uniqueConstraints =
        @UniqueConstraint(columnNames = "USERNAME"))
public class User {
    @Override
    public boolean equals(Object other) {
        if (this == other) return true;
        if (other == null) return false;
        if (!(other instanceof User)) return false;
        User that = (User) other;
        return this.getUsername().equals(that.getUsername());
    }
    @Override
    public int hashCode() {
        return getUsername().hashCode();
    }
    //  . . .
}
```

You may have noticed that the `equals()` method code always accesses the properties of the "other" reference via getter methods. This is extremely important because the reference passed as `other` may be a Hibernate proxy, not the actual instance that holds the persistent state. We can't access the `username` field of a `User` proxy directly. To initialize the proxy to get the property value, we need to access it with a getter method. This is one point where Hibernate isn't *completely* transparent, but it's good practice anyway to use getter methods instead of direct instance variable access.

Check the type of the `other` reference with `instanceof`, rather than by comparing the values of `getClass()`. Again, the `other` reference may be a proxy, which is a runtime-generated subclass of `User`, so `this` and `other` may not be exactly the same type but a valid supertype or subtype. You'll learn more about proxies in section 12.1.1.

We can now safely compare `User` references in persistent state:

```
em = emf.createEntityManager();
em.getTransaction().begin();
User a = em.find(User.class, USER_ID);
User b = em.find(User.class, USER_ID);
assertTrue(a == b);
assertTrue(a.equals(b));
assertEquals(a.getId(), b.getId());
em.getTransaction().commit();
em.close();
```

We also, of course, get correct behavior if we compare references to instances in the persistent and detached state:

```
em = emf.createEntityManager();
em.getTransaction().begin();
User c = em.find(User.class, USER_ID);
assertFalse(a == c);                          Ⓐ
assertTrue(a.equals(c));                      Ⓑ
assertEquals(a.getId(), c.getId());
em.getTransaction().commit();
em.close();
Set<User> allUsers = new HashSet();
allUsers.add(a);
allUsers.add(b);
allUsers.add(c);
assertEquals(1, allUsers.size());             Ⓒ
```

Ⓐ Comparing the two references will, of course, still be false.

Ⓑ Now they are equal.

Ⓒ The size of the set is finally correct.

For some other entities, the business key may be more complex, consisting of a combination of properties. Here are some hints that should help you identify a business key in the domain model classes:

- Consider what attributes users of the application will refer to when they have to identify an object (in the real world). How do users tell the difference between one element and another if they're displayed on the screen? This is probably the business key to look for.
- Every immutable attribute is probably a good candidate for the business key. Mutable attributes may be good candidates, too, if they're updated rarely or if you can control the case when they're updated, such as by ensuring the instances aren't in a `Set` at the time.
- Every attribute that has a `UNIQUE` database constraint is a good candidate for the business key. Remember that the precision of the business key has to be good enough to avoid overlaps.
- Any date or time-based attribute, such as the creation timestamp of the record, is usually a good component of a business key, but the accuracy of `System .currentTimeMillis()` depends on the virtual machine and operating system. Our recommended safety buffer is 50 milliseconds, which may not be accurate enough if the time-based property is the single attribute of a business key.

- You can use database identifiers as part of the business key. This seems to contradict our previous statements, but we aren't talking about the database identifier value of the given entity. You may be able to use the database identifier of an associated entity instance. For example, a candidate business key for the `Bid` class is the identifier of the `Item` it matches, together with the bid amount. You may even have a unique constraint that represents this composite business key in the database schema. You can use the identifier value of the associated `Item` because it never changes during the lifecycle of a `Bid` — the `Bid` constructor can require an already-persistent `Item`.

If you follow this advice, you shouldn't have much difficulty finding a good business key for all your business classes. If you encounter a difficult case, try to solve it without considering Hibernate. After all, it's purely an object-oriented problem. Notice that it's extremely rarely correct to override `equals()` on a subclass and include another property in the comparison. It's a little tricky to satisfy the `Object` identity and equality requirements that equality is both symmetric and transitive in this case, and more important, the business key may not correspond to any well-defined candidate natural key in the database (subclass properties may be mapped to a different table). For more information on customizing equality comparisons, see *Effective Java*, third edition, by Joshua Bloch (Bloch, 2017), a mandatory book for all Java programmers.

The `User` class is now prepared for the detached state; we can safely put instances loaded in different persistence contexts into a `Set`. Next we'll look at some examples that involve the detached state, and you'll see some of the benefits of this concept.

10.3.3 Detaching entity instances

Sometimes we might want to detach an entity instance manually from the persistence context. We don't have to wait for the persistence context to close. We can evict entity instances manually:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java – detach()

User user = em.find(User.class, USER_ID);
em.detach(user);
assertFalse(em.contains(user));
```

This example also demonstrates the `EntityManager#contains()` operation, which returns `true` if the given instance is in the managed persistent state in this persistence context.

We can now work with the `user` reference in a detached state. Many applications only read and render the data after the persistence context is closed.

Modifying the loaded `user` after the persistence context is closed has no effect on its persistent representation in the database. JPA allows us to merge any changes back into the database in a new persistence context, though.

10.3.4 Merging entity instances

Let's assume we've retrieved a `User` instance in a previous persistence context, and now we want to modify it and save these modifications:

```
Path: managing-data/src/test/java/com/manning/javapersistence/ch10
➥ /SimpleTransitionsTest.java — mergeDetached()

detachedUser.setUsername("johndoe");
em = emf.createEntityManager();
em.getTransaction().begin();
User mergedUser = em.merge(detachedUser);
mergedUser.setUsername("doejohn");
em.getTransaction().commit();
em.close();
```

Consider the graphical representation of this procedure in figure 10.5. The goal is to record the new `username` of the detached `User`. It's not as difficult as it seems.

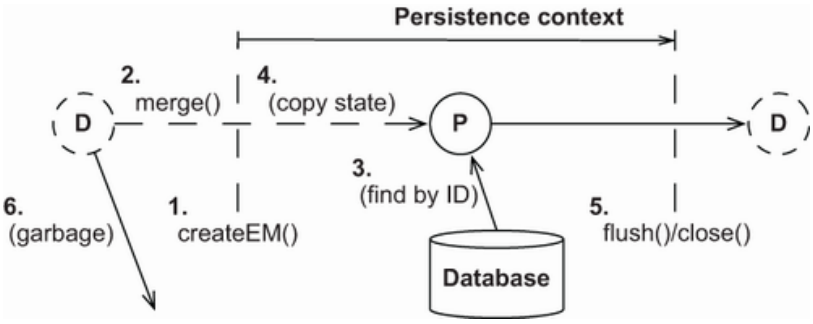

Figure 10.5 Making an instance persistent in a unit of work

First, when we call `merge()`, Hibernate checks whether a persistent instance in the persistence context has the same database identifier as the detached instance we merging. In this example, the persistence context is empty; nothing has been loaded from the database. Hibernate, therefore, loads an instance with this identifier from the database. Then `merge()` copies the detached entity instance *onto* this loaded persistent instance. In other words, the new `username` we have set on the detached `User` is also set on the persistent merged `User`, which `merge()` returns to us.

Now we discard the old reference to the stale and outdated detached state; the `detachedUser` no longer represents the current state. We can continue modifying the returned `mergedUser`; Hibernate will execute a single `UPDATE` when it flushes the persistence context during commit.

If there is no persistent instance with the same identifier in the persistence context, and a lookup by identifier in the database is negative, Hibernate instantiates a fresh `User`. Hibernate then copies our detached instance onto this fresh instance, which it inserts into the database when we synchronize the persistence context with the database.

If the instance we're giving to `merge()` is not detached but rather is transient (it doesn't have an identifier value), Hibernate instantiates a fresh `User`, copies the values of the transient `User` onto it, and then makes it persistent and returns it to us. In simpler terms, the `merge()` operation can handle detached *and* transient entity instances. Hibernate always returns the result to us as a persistent instance.

An application architecture based on detachment and merging may not call the `persist()` operation. We can merge new and detached entity instances to store data. The important difference is the returned current state and how we handle this switch of references in our application code. We have to discard the `detachedUser` and from now on reference the current `mergedUser`. Every other component in our application still holding on to `detachedUser` has to switch to `mergedUser`.

Can I reattach a detached instance?

The Hibernate `Session` API has a method for reattachment called `saveOrUpdate()`. It accepts either a transient or a detached instance and doesn't return anything. The given instance will be in a persistent state after the operation, so we don't have to switch references. Hibernate will execute an `INSERT` if the given instance was transient or an `UPDATE` if it was detached. We recommend that you rely on merging instead, because it's standardized and therefore easier to integrate with other frameworks. In addition, instead of an `UPDATE`, merging may only trigger a `SELECT` if the detached data wasn't modified. If you're wondering what the `saveOrUpdateCopy()` method of the `Session` API does, it's the same as `merge()` on the `EntityManager`.

If we want to delete a detached instance, we have to merge it first. Then we can call `remove()` on the persistent instance returned by `merge()`.

Summary

- The lifecycle of entity instances includes the transient, persistent, detached, and removed states.
- The most important interface in JPA is `EntityManager`.
- We can use the `EntityManager` to make data persistent, retrieve and modify persistent data, get a reference, make data transient, refresh and replicate data, cache in the persistence context, and flush the persistence context.
- We can work with the detached state, using the identity of detached instances and implementing equality methods.